

L05 Team Report: SunFounder PiCar-X Robot

Participating Team Members: Andre, Eva, Muhammad

The team members listed above vouch that everyone listed participated appropriately in this assignment.

1. Introduction and Objectives

The objective of these lab sessions was to initialize the SunFounder Picar-X robot using a Raspberry Pi 5 and to verify its hardware functionality through Python script execution. Since the robot was already assembled, our focus was on the software integration and system configuration required to make the unit operational.

During the initial assessment, we identified a permanent hardware limitation with the unit's camera module. Consequently, we focused our objectives on system stability, I2C interface configuration, and motor calibration. This report details the commands used and the troubleshooting steps we took to move from initial boot-up to successful locomotion.

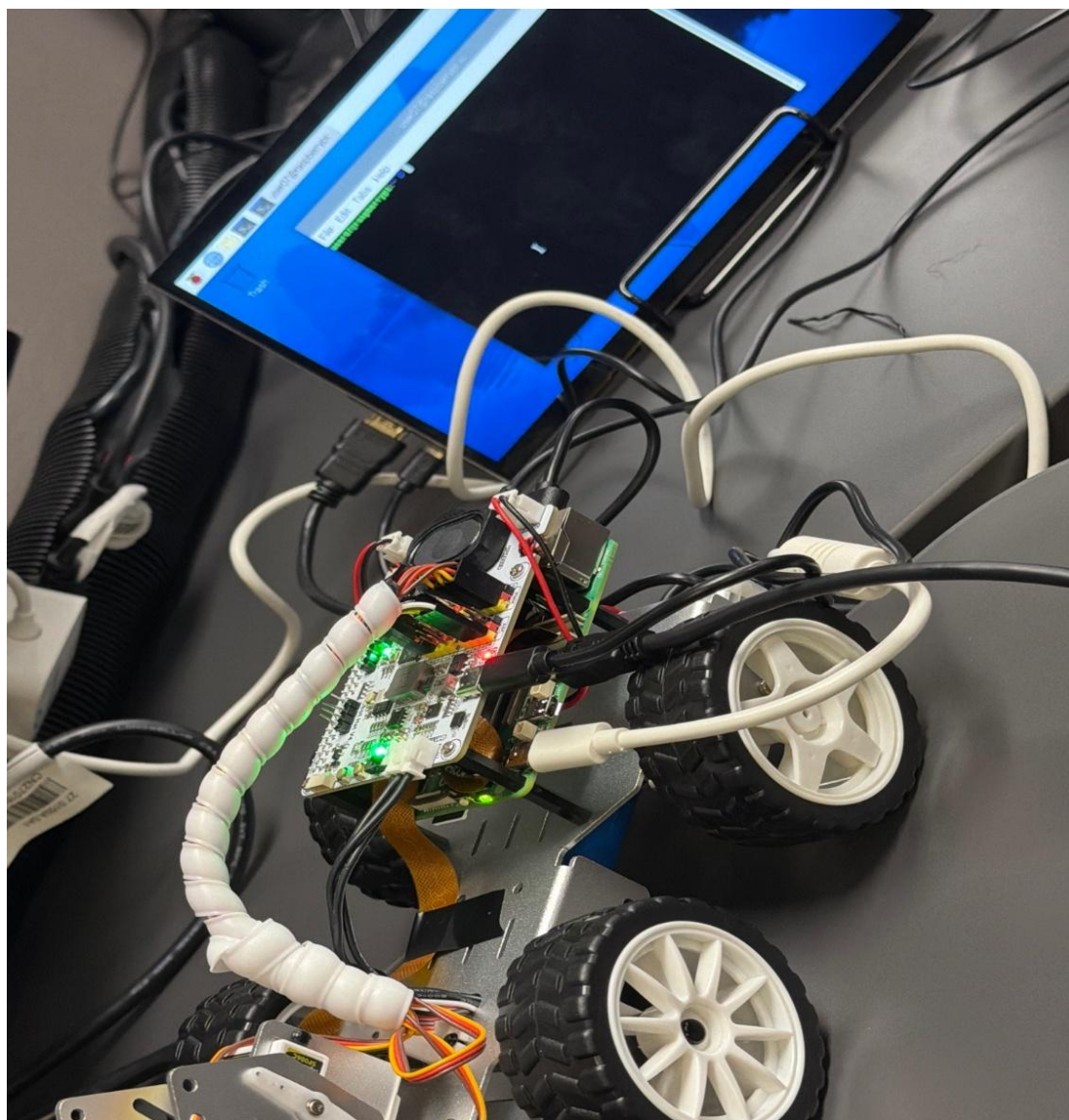
2. Activity Description

Our team followed a step-by-step process to configure the Raspberry Pi 5 environment for the Picar-X hardware.

System Initialization

We connected the Raspberry Pi 5 to a portable monitor to begin the setup. We encountered an immediate obstacle when the keyboard failed to respond due to dead batteries. After replacing the keyboard, we were able to access the terminal and begin the configuration process.

Figure 1: Initial workstation setup and hardware connection



Interface Configuration

To allow the Pi to communicate with the robot's motor hat, we had to enable the I2C interface. We executed `sudo raspi-config` to access the system tool. As shown in **Figure 2a**, we entered the **Interface Options** menu. From there, we successfully enabled the I2C setting (**Figure 2b**). This was a critical step, as I2C is the primary protocol used to send movement commands to the hardware.

Figure 2a: Navigating the raspi-config tool

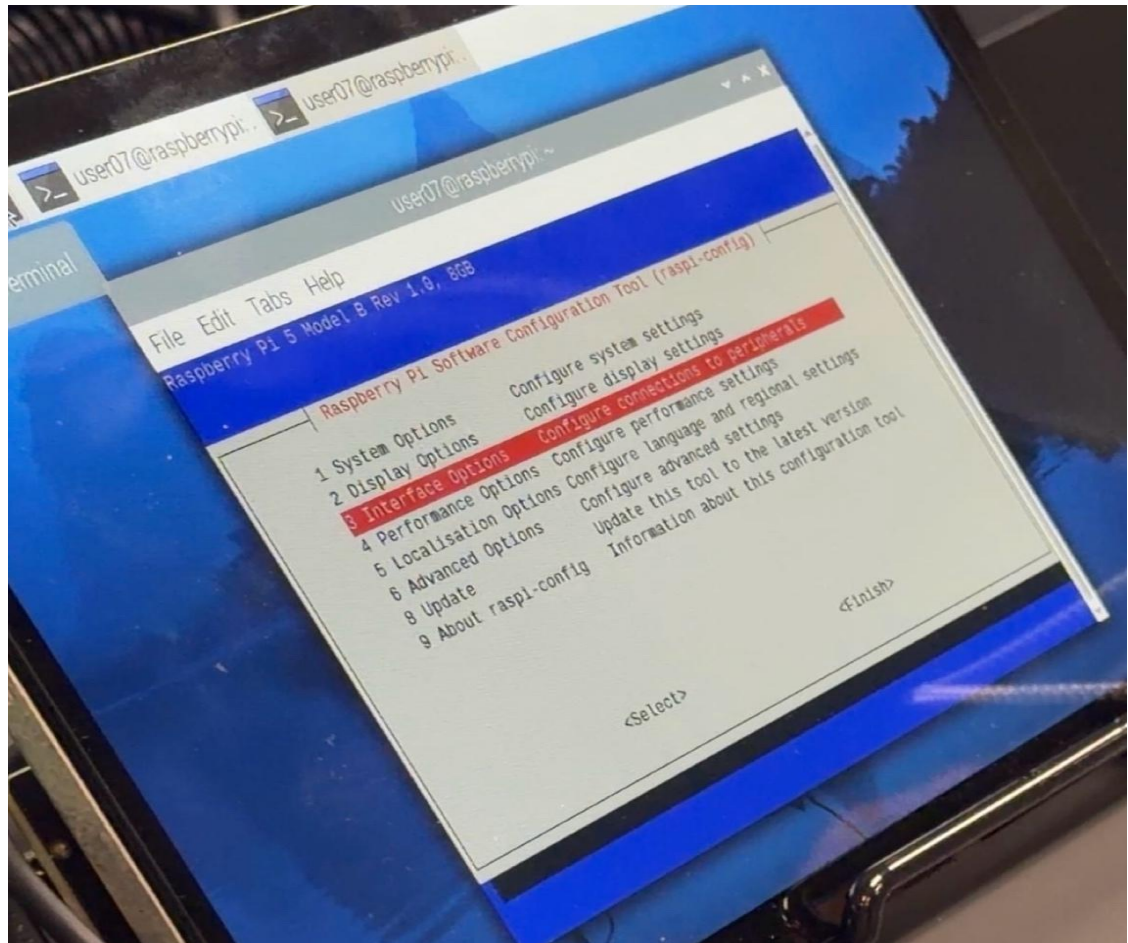
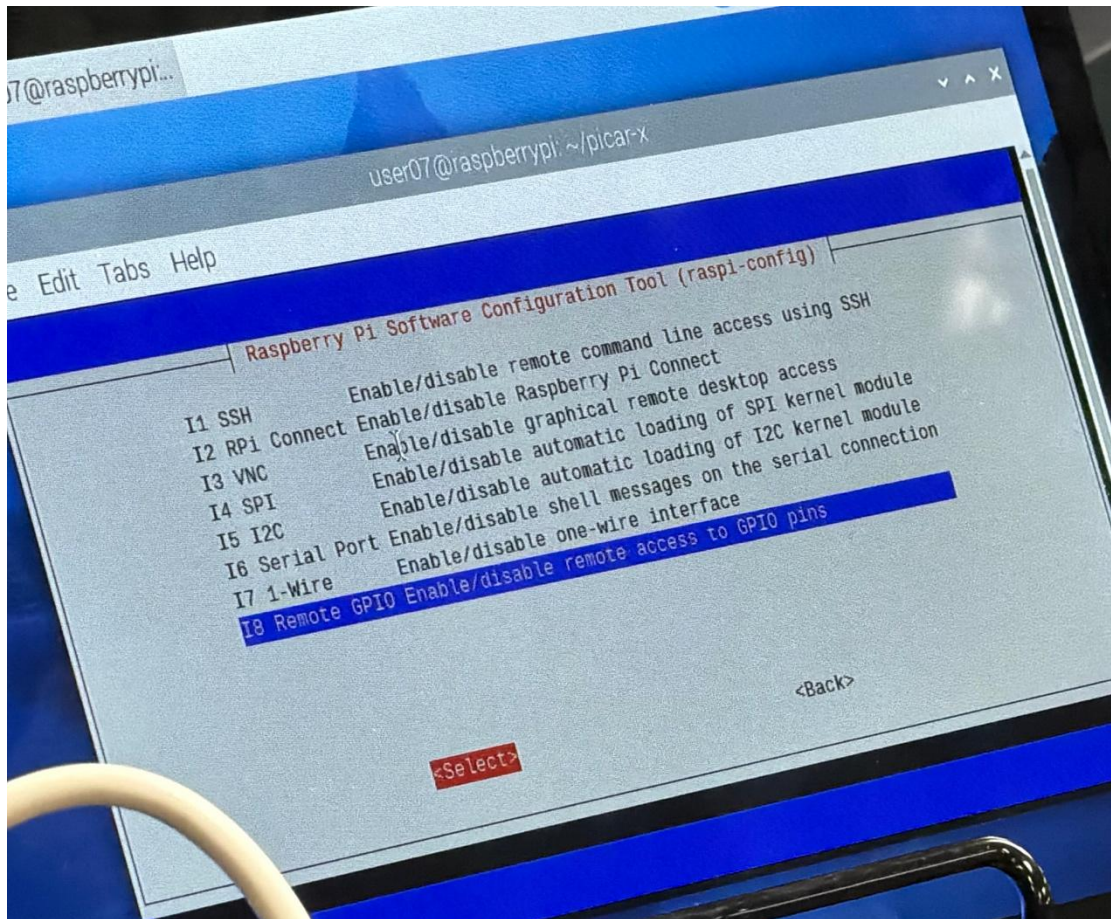


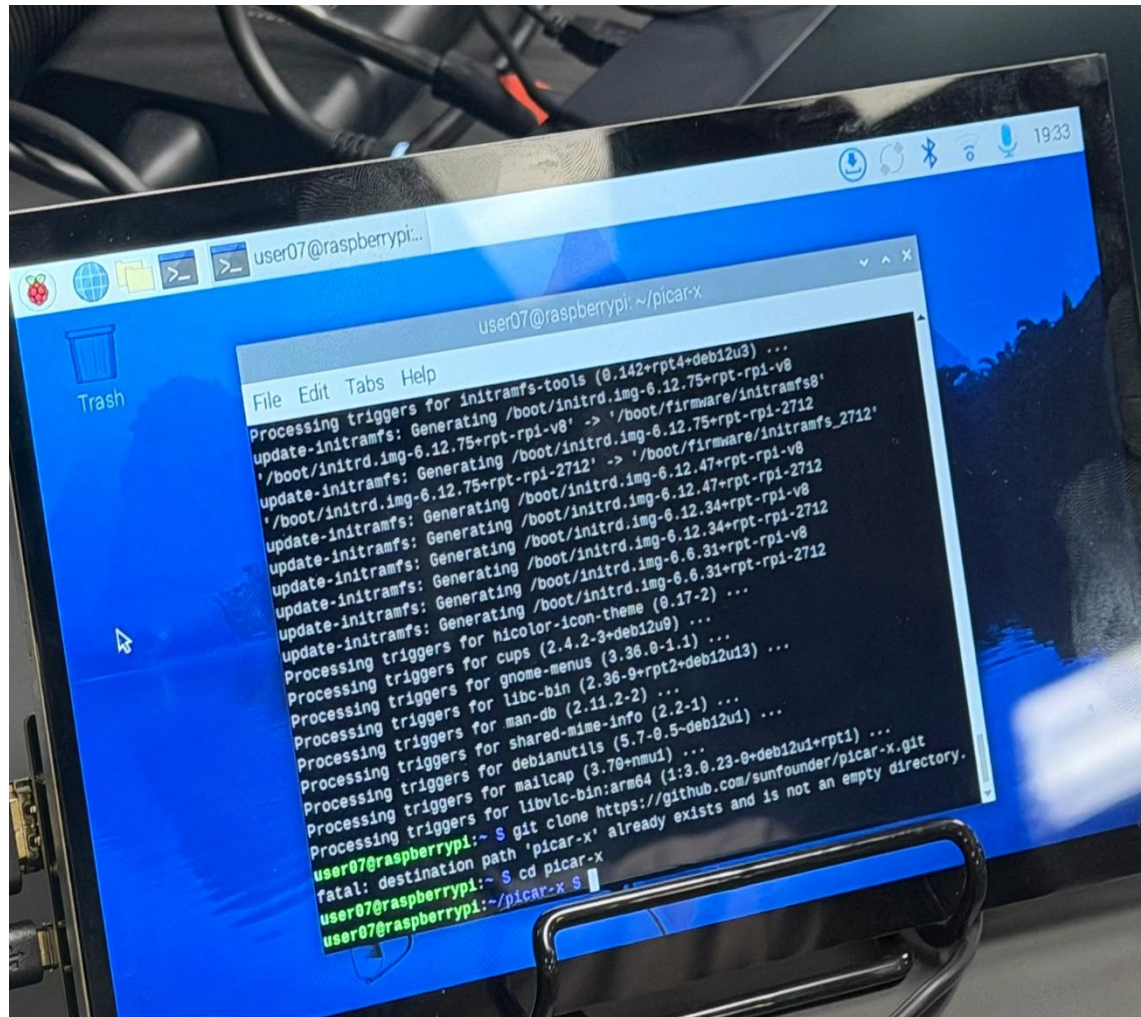
Figure 2b: Enabling the I2C interface for hardware communication

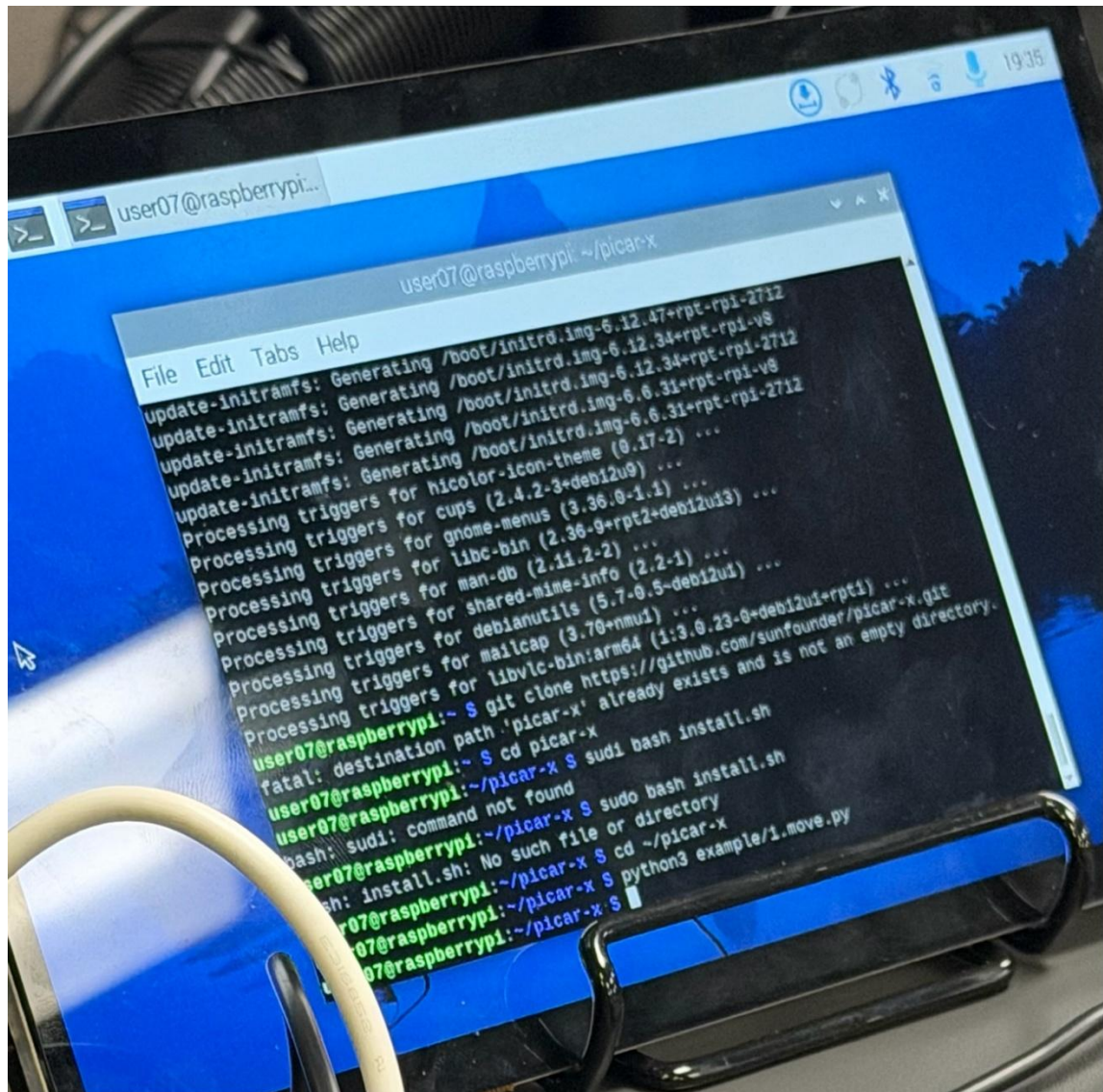


Software Installation

Once the hardware interfaces were configured, we proceeded to update the operating system using `sudo apt update` and `sudo apt upgrade -y`. **Figure 3a** illustrates the terminal during this process, showing the generation of the **Initial RAM File System (initramfs)** boot files. This was a necessary step to ensure the system's pre-boot environment was updated alongside the new software. After the system was current, we navigated to the robot's software directory. As shown in **Figure 3b**, our initial attempt to clone the Picar-X repository confirmed the directory already existed on the local storage. We then successfully changed directories into the `picar-x` folder to begin the library installation via the `install.sh` script.

Figure 3a: Monitoring system triggers and Initial RAM File System generation during the upgrade





3. Code Documentation

To verify the robot's hardware functionality, our team utilized the pre-installed Python example scripts. Before execution, we used the **GNU nano** text editor to review the script logic; this allowed us to ensure that the GPIO (General Purpose Input/Output) pins and motor constants were correctly defined for our specific hardware revision.

Primary Execution Commands:

```
# Navigating to the examples directory and executing the movement script
```

```
cd ~/picar-x/example  
python3 1.move.py
```

Figure 4: Reviewing the Python source code within the GNU nano editor to verify locomotion logic

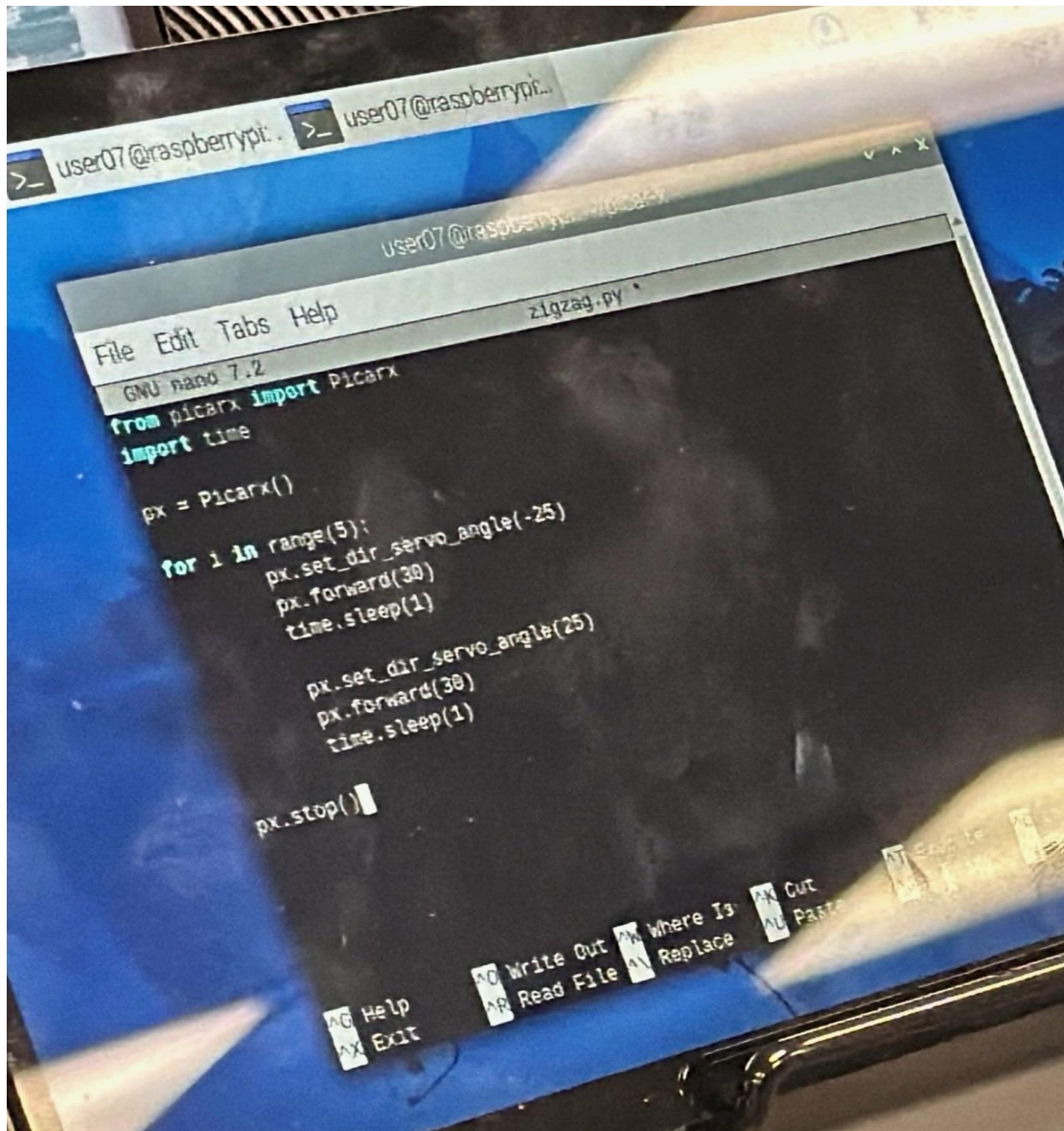


Figure 5: Terminal command used to initiate the locomotion test

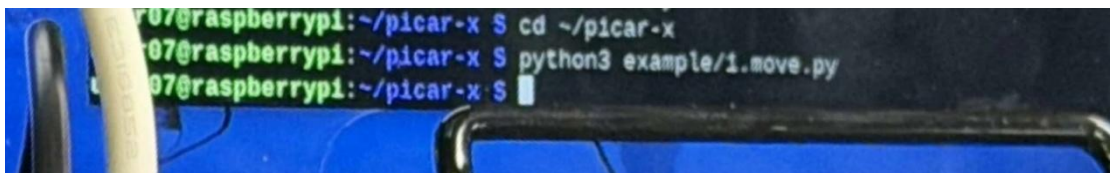


Figure 6: Video demonstration of the Picar-X robot executing the programmed movement script. Click here to view the video:

<https://youtu.be/akiykhMHB48?si=KycAuc5LAsj20gpP>

4. Obstacles and Troubleshooting

This section details the technical challenges encountered during the assembly and configuration of the Picar-X and the corrective actions taken.

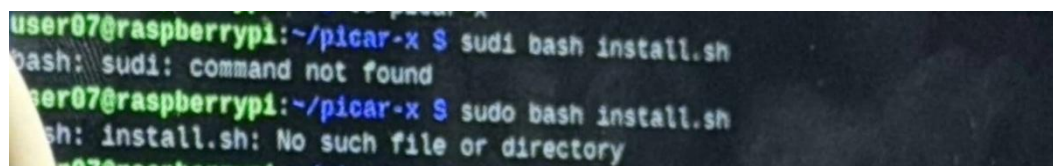
a. Hardware Power & Peripherals

Our first obstacle involved the physical setup. We discovered that the initial keyboard we attempted to use was unresponsive due to **depleted batteries**, requiring a quick swap to a secondary unit to continue the installation. Additionally, we realized mid-lab that the **Picar-X and its battery packs** needed to be fully charged to ensure consistent power delivery to the motors and the Raspberry Pi 5.

b. Software Syntax & Navigation

On the software side, we encountered several “command not found” errors. These were largely due to minor syntax typos (e.g., typing `sudi` instead of `sudo`) and directory pathing issues. As seen in **Figure 7**, we learned that commands must be executed from the specific `~/picar-x` directory for the Python scripts to find the necessary local libraries.

Figure 7: Close-up of terminal syntax error and directory pathing issues



c. Camera Status

After confirming the robot’s movement, we attempted to test the visual systems. However, we determined that our camera module was out of commission. Despite checking the physical ribbon cable connections and software configurations, the hardware remained unresponsive. Because the camera was non-functional, we pivoted our focus to perfecting the locomotion and ultrasonic sensor logic to maximize the value of our lab time.

d. Collaborative Debugging (Video Log):

Throughout the troubleshooting process, we maintained a video log of our progress. The extended footage captures the team identifying hardware issues and working through terminal errors in real-time.

Figure 8: Lab documentation showing the collaborative debugging process. Click here to watch the video:

<https://youtu.be/uNzK5V7VJ8w?si=GMOHYSJY0Tue96We>

5. Results and Accomplishments

The primary objective of the lab –successfully interfacing the Raspberry Pi 5 with the Picar-X chassis—was achieved. While we had to adapt to hardware limitations, specifically the non-functional camera, the final tests confirmed that the core systems of the robot are fully operational and responsive to our custom software environment.

a. Locomotion and Steering Calibration

The most significant result was the successful execution of the 1.move.py script. The robot did not just move forward; it demonstrated precise control over its steering servos and drive motors.

Maneuverability: In our final demonstration, the Picar-X showed smooth navigation and the ability to steer around obstacles (like our laptop) on the workbench.

Command Execution: The transition from the Python script to physical movement was instantaneous, proving that our library configurations and HAT connections were correct.

Figure 9: Final performance demonstration showcasing integrated steering, drive motor synchronization, and successful manual obstacle avoidance. Click the link to view the video:

https://youtube.com/shorts/ul3ReHcA_SI?feature=share

b. Software Environment & Mobile Setup

We successfully built a stable workspace that allowed us to work efficiently in a lab setting.

OS Readiness: By completing the sudo apt updates and upgrades, we ensured the Raspberry Pi 5 was fully compatible with the Picar-X's specific Python requirements.

Mobile Debugging: Utilizing the portable HDMI monitor allowed us to keep the terminal visible during physical movement tests. This setup was a key accomplishment because it allowed us to see real-time errors and confirm successful script execution without being tethered to a stationary monitor.

c. Summary of Project Milestones

The following table summarizes the functional status of the Picar-X system at the conclusion of the lab:

Objective	Status	Result
Basic Locomotion	Successful	Robot executes forward/reverse commands.
Steering Control	Successful	Front-wheel servos are calibrated and turning.
System Stability	Successful	Pi 5 and HAT remain powered during high-torque movement.
Vision Systems	Incomplete	Hardware (Camera) confirmed out of commission.

6. What we learned

This lab provided several key insights into the integration of hardware and software within a robotics framework. Beyond the basic assembly, the team walked away with the following takeaways:

- **The importance of Precise Syntax**

We learned that when working in a Linux-based environment, there is zero room for error in the terminal. A single character typo (like `sudi` vs `sudo`) or a missing directory level can halt an entire project. This reinforced the habit of double-checking commands and using the `ls` command frequently to verify our current path before executing scripts.

- **Power Management as a Performance Factor**

A major takeaway was the direct relationship between battery health and hardware performance. We realized that even if the Raspberry Pi 5 is powered on, low voltage in the Picar-X battery packs can cause erratic motor behavior or script crashes. Moving forward, we understand that “full charge” is a prerequisite for any hardware testing to ensure data consistency.

- **Hardware Integrity and Adaptability**

This lab taught us how to handle equipment failure professionally. Discovering a faulty camera module could have been a major setback, but we learned how to “pivot”—reallocating our time to perfect the locomotion and steering logic instead. We learned that in engineering, you must often work with the hardware you have, not the hardware you wish you had.

- **Collaborative Debugging**

Finally, we learned the value of “multiple sets of eyes” on a problem. Most of our software hurdles were solved through peer review, where one team member could catch a typo that another had missed. This collaborative approach significantly reduced our downtime and made the troubleshooting process much more efficient.

7. Sources of Information

We utilized a blend of official documentation and collaborative tools to complete the Picar-X configuration and troubleshooting:

Course Materials: *Computing using the Raspberry Pi 05* was used for Linux terminal fundamentals, Raspberry Pi 5 pinout references, and environment setup.

Technical Guides: *The SunFounder Picar-X Online User Guide* served as the primary reference for Python library structures and repository navigation.

Generative AI (ChatGPT): Employed as a real-time debugging consultant to interpret complex Linux error logs and validate Python 3 command syntax.

Peer Collaboration: Technical exchanges with classmates were used to cross-reference hardware performance and verify that the camera module failure was an isolated hardware defect.